

Logistic Regression — Short Notes

What is Logistic Regression?

Logistic Regression is a supervised learning algorithm used for classification, mainly for predicting categories such as:

Student → Placed / Not Placed

Email → Spam / Not Spam

Patient → Disease / No Disease

Unlike Linear Regression, its output is a **probability between 0 and 1**.

Basic Idea

First calculate:

Then pass z through the **Sigmoid function**:

This converts the output into a probability.

Example:

Sigmoid output = 0.85

↓

85% probability of Class 1

↓

Prediction = Class 1

Decision Boundary

A boundary separates the classes:

Class 0 | Decision Boundary | Class 1

For two features, it's a **line**; with more features, it's a **hyperplane**.

Perceptron Trick

The perceptron starts with random weights and:

1. Makes a prediction.
2. Checks whether it is wrong.
3. Updates the weights.
4. Repeats.

A simplified update is:

where:

- $\eta \rightarrow$ learning rate

- $y \rightarrow$ actual label
- $\hat{y} \rightarrow$ prediction
- $x \rightarrow$ input

Remember

Input features



Weighted sum (z)



Sigmoid



Probability (0 to 1)



Class prediction

Logistic Regression = Linear equation + Sigmoid $\rightarrow$ Classification.

Perceptron Trick — Short Notes

How it works

1. Add a **1 column** to $X \rightarrow$ represents the **bias/intercept**.
2. Initialize weights.
3. Pick a training point.
4. Make a prediction.
5. If prediction is wrong $\rightarrow$ **update weights**.
6. Repeat for several epochs.

Update

- $\eta \rightarrow$ learning rate
- $y \rightarrow$ actual class
- $\hat{y} \rightarrow$ predicted class
- $x \rightarrow$ input features

Main Limitation

The Perceptron stops when it can classify the training data correctly.

Problem: It doesn't necessarily find the **best decision boundary**.

So it can give:

Perceptron



Correctly classifies training data



But boundary may be poor



Poor generalization

Perceptron vs Logistic Regression

Perceptron

Focuses on correct classification

Can stop at any separating boundary

No probability output

Mainly useful for learning

Logistic Regression

Optimizes probability/loss

Finds a more suitable boundary

Gives probability

Widely used in practice

Remember

Perceptron says: "Just separate the classes."

Logistic Regression says: "Find a good boundary and estimate probabilities."

Sigmoid Function — Short Notes

Why Sigmoid?

The **Perceptron** uses a **step function**:

$$z < 0 \rightarrow 0$$

$$z \geq 0 \rightarrow 1$$

Problem: it only gives **0 or 1**, so there is no gradual change to help update the model.

Sigmoid Function

Logistic Regression uses:

It converts any value into a **0–1 probability**.

$z \rightarrow \text{Sigmoid} \rightarrow \text{Probability}$

Example:

$$z = -2 \rightarrow 0.12$$

$$z = 0 \rightarrow 0.50$$

$$z = 2 \rightarrow 0.88$$

So the model can say:

"There is an **88% probability** this belongs to **Class 1**."

Why is this better than Perceptron?

Instead of:

Correct → No update

Wrong → Update

we get a **continuous probability**, so the model can measure **how confident** it is and make gradual updates.

 **Remember**

Perceptron → Step function → 0/1

Logistic Regression → Sigmoid → 0–1 probability

The **Sigmoid** is the key bridge from the Perceptron idea to Logistic Regression.

Logistic Regression — Loss Function

1. Why do we need a Loss Function?

The **Perceptron** only tells us whether the prediction is correct or incorrect.

But Logistic Regression gives a **probability**, so we need to measure **how good that probability is**.

Example:

Actual class = 1

Prediction = 0.95 → Very good

Prediction = 0.60 → Not very confident

Prediction = 0.01 → Very bad

A loss function gives a **large penalty for bad predictions** and a **small penalty for good predictions**.

2. Maximum Likelihood Estimation (MLE)

MLE is used to find the model parameters that make the **observed data most likely**.

Suppose the actual labels are:

$y = [1, 0, 1]$

Model probabilities:

$P = [0.9, 0.8, 0.7]$

For a class 1, we want a **high probability**.

For a class 0, we want a **low probability**.

So the likelihood is:

General form:

We want to **maximize this likelihood**.

3. Why use Log?

When we have thousands of data points, multiplying many probabilities can produce an extremely tiny number.

For example:

becomes very small.

So we take **logarithm**.

Because:

Multiplication becomes addition, which is much easier to calculate.

4. From MLE to Cross-Entropy

We want to:

But ML optimization is usually written as **minimization**.

So we multiply by -1:

This gives **Log Loss / Binary Cross-Entropy**.

5. Binary Cross-Entropy Formula

Where:

- = actual label (0 or 1)
 - = predicted probability
 - = number of observations
-

6. Understand it with one example

Suppose:

Actual $y = 1$

Predicted probability = 0.9

Loss:

≈ 0.105

Very small → **good prediction**.

Now:

Actual $y = 1$

Predicted probability = 0.1

Loss:

≈ 2.303

Large → **bad prediction**.

So Cross-Entropy strongly penalizes **confident wrong predictions**.

7. What if actual class is 0?

Suppose:

Actual $y = 0$

Predicted probability = 0.1

The loss uses:

≈ 0.105

Good prediction.

But if:

Actual $y = 0$

Predicted probability = 0.9

Then:

≈ 2.303

Very bad prediction.

8. Why can't we use MSE?

You *can* mathematically use MSE in some settings, but for Logistic Regression, **Binary Cross-Entropy is preferred** because it works naturally with probabilities and gives a suitable optimization landscape for classification.

9. Why Gradient Descent?

For Linear Regression, we can derive a direct OLS solution.

For Logistic Regression with Cross-Entropy, there is **no simple closed-form solution** for the optimal weights.

Therefore:

Features



Linear equation



Sigmoid



Probability



Cross-Entropy Loss



Gradient Descent



Update weights



Repeat



The complete connection

MLE finds parameters that make the observed data most likely. Taking the negative log of the likelihood gives us Binary Cross-Entropy Loss. Logistic Regression minimizes this loss using Gradient Descent to find the optimal weights.

That's the important chain to remember:

MLE → Log Likelihood → Negative Log → Cross-Entropy → Gradient Descent → Optimal weights.

Sigmoid Derivative — Notes

1. Sigmoid Function

The sigmoid function is:

It converts any value into a range between **0 and 1**.

2. Why do we need its derivative?

The derivative tells us **how much the sigmoid output changes when the input changes**.

This is important for **Gradient Descent and neural network training**, because we need derivatives to update the weights.

3. Derivation

Start with:

Rewrite:

Using the **chain rule**:

becomes:

Therefore:

Now comes the important simplification.

Since:

and:

we get:

4. Example

If:

then:

So the derivative is **0.16**.



Most important thing

Memorize this: $\text{Sigmoid derivative} = \text{Sigmoid} \times (1 - \text{Sigmoid})$

Classification Metrics — Notes

1. Accuracy

Accuracy tells us **how many predictions were correct out of all predictions**.

$$\left[\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \right]$$

Example:

100 predictions → 90 correct

$$\left[\text{Accuracy} = 90\% \right]$$

Problem: Accuracy can be misleading with **imbalanced data**.

2. Confusion Matrix

It shows **what type of predictions the model made**.

	Actual 1	Actual 0
Predicted 1	TP	FP
Predicted 0	FN	TN

- **TP:** Actual 1 → Predicted 1 ✓
- **TN:** Actual 0 → Predicted 0 ✓
- **FP:** Actual 0 → Predicted 1 ✗ (Type 1 Error)
- **FN:** Actual 1 → Predicted 0 ✗ (Type 2 Error)

Example: Disease Detection

Actual: Disease

Prediction: Disease → TP ✓

Actual: Healthy

Prediction: Disease → FP ✗

Actual: Disease

Prediction: Healthy → FN ✗

Actual: Healthy

Prediction: Healthy → TN ✓

3. Why Accuracy Can Fail

Suppose there are **1000 people**:

990 → Healthy

10 → Disease

A useless model predicts **Healthy for everyone**.

It gets:

$990 / 1000 = 99\%$ accuracy

But it detects **0 out of 10 sick people**. ✗

So accuracy says **99%**, while the model is actually useless for the important class.

That's why we use:

Precision, Recall, F1-score, ROC-AUC, etc.

Remember

Accuracy = How many were correct?

Confusion Matrix = What kinds of mistakes did the model make?

Classification Metrics — Short Notes

1. Precision

Precision: Out of all predictions made as **Positive**, how many were actually Positive?

$$\left[\begin{array}{l} \text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \end{array} \right]$$

Use when False Positives are costly.

Example: **Spam detection** → Don't mark important emails as spam.

2. Recall

Recall: Out of all **actual Positive** cases, how many did the model correctly detect?

$$\left[\begin{array}{l} \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \end{array} \right]$$

Use when False Negatives are costly.

Example: **Disease detection** → Don't miss a sick patient.

3. F1 Score

F1 combines **Precision and Recall** using their harmonic mean:

$$\left[\begin{array}{l} \text{F1} = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \end{array} \right]$$

If either Precision or Recall is low, **F1 also becomes low**.

Use when you need a balance between Precision and Recall.

4. Multi-Class Classification

When there are more than 2 classes, metrics are calculated **for each class**.

Macro Average:

Gives equal importance to every class.

Weighted Average:

Gives more importance to classes having more samples.

Easy Way to Remember

Precision → "When I said Positive, was I correct?"



Avoid FP

Recall → "Did I find all the actual Positives?"



Avoid FN

F1 → "Balance between Precision + Recall"

Scikit-Learn

```
from sklearn.metrics import precision_score, recall_score, f1_score
```

```
precision_score(y_test, y_pred)
```

```
recall_score(y_test, y_pred)
```

```
f1_score(y_test, y_pred)
```

Or get everything together:

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, y_pred))
```

Key idea:

Precision cares about False Positives; Recall cares about False Negatives; F1 balances both.
